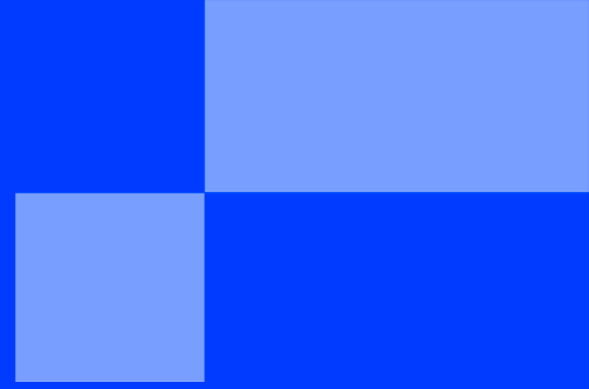




SUMMER SCHOOL

< PART 3 - THE INTERACTIVE NEWSPAPER />

SUMMER SCHOOL

< WEB DEVELOPMENT: IT IS ALIVE! />

"Design is not just what it looks like and feels like. Design is how it works."
— **Steve Jobs**

The Case

The good news: The Editor-in-Chief loves the new React architecture. The code is clean, the components are modular, and the visual design is stunning.

The bad news: The Marketing Director just stormed into your office. "It is a beautiful painting, but it is completely dead! We have zero engagement metrics, and the search bar is a fake! Static pages are so 2005!"

Your mission today is to breathe life into your Frankenstein monster. It is time to move from a passive display to an interactive web application. The users need to click, type, and engage.

Project Requirements

You will be working on the state management of your React application. This will allow your components to remember information and react to user interactions.

1. Feature: The Engagement Metric (Likes)

The marketing team needs to know which articles are performing best to sell advertising space.

- Inside your `ArticleCard` component, add an interactive "Like" button.
- Display the current number of likes directly on the card next to the button.
- Every time a user clicks the button, the counter for *that specific article* must increase by 1.
- **Acceptance Criteria:** Clicking "Like" on the breaking news must NOT increase the likes on the other smaller articles. Each component must remember its own state independently.

2. Feature: The Functional Search Bar

The editorial team is complaining that the archive search at the bottom of the page is just a decoration.

- Upgrade the `<input>` inside your `Footer` component to be a "Controlled Component".
- The application must know exactly what the user is typing in real-time.
- Upgrade the "Search" `<button>`. When clicked, it must trigger an action (an `alert()` or a `console.log()`) that says: *"Searching archives for: [User's Input]"*.
- **Acceptance Criteria:** If the user clicks "Search" while the input is empty, display an error message (e.g., *"Please enter a search term!"*) instead of executing the empty search.

Bonus (Optional)

If you finished early and want to impress the CTO, try implementing these advanced features:

- ✓ **The Toggle (Unlike):** Change the "Like" button behavior. If a user clicks it, it adds a like. If they click it again, it removes their like. A single user cannot like an article 50 times!
- ✓ **Live Character Counter:** Underneath the search bar in the footer, display a live counter showing how many characters the user has typed (e.g., "12/50 characters").
- ✓ **The Dark Mode Switch:** Add a button in your `Header` that toggles a `dark-mode` CSS class on the main `<body>` or main container. This is the ultimate test of state management and CSS styling!

v1

{EPITECH}